

第6回：意味で探す検索

Embeddingと類似度を理解する

今日のゴールと流れ

文章のベクトル化と、検索の比較を体験する。

時間	内容
10分	キーワード検索の失敗の復習
20分	Embeddingと類似度
20分	登録処理と質問時の検索
30分	計算の穴埋め・検索の比較
10分	更新・費用・提出

言い方が違う質問

質問：「ページを閉じててもデータが残るのはなぜ？」

資料：「SQLiteで永続化する」

同じ単語がなくても、関連する文章として探したい場面があります。

Embeddingモデルで文章を数値の列へ変換し、近さを比べます。

必ず見つかるわけではありません。第5回の結果と比較します。

ベクトルとは

数値を並べたものです。説明用の例：

```
a = [1.0, 0.0, 1.0]  
b = [0.9, 0.1, 1.0]  
c = [0.0, 1.0, 0.0]
```

実際のEmbeddingは、多数の次元を持ちます。

各数値を「価格」「色」のように直接解釈できるとは限りません。

生成用と検索用のモデル

用途	モデル	出力
回答を作る	gpt-6-luna	文章など
文章をベクトル化	text-embedding-3-small	数値の列

資料と質問には、同じEmbeddingモデルを使います。
別モデルの座標同士を、そのまま比較しません。

2つのタイミング

資料の登録・更新時	質問を受けたとき
断片を1つずつベクトル化	質問をベクトル化
ベクトルとモデル名を保存	保存済みの各ベクトルと比較
次の質問で再利用	上位の本文を取得して生成へ

資料のベクトルを毎回作り直す必要はありません。

Embedding API

```
response = client.embeddings.create(  
    model=EMBEDDING_MODEL,  
    input=text,  
)  
vector = response.data[0].embedding
```

第2回と同じクライアントを使いますが、呼ぶAPIが異なります。
この呼び出しでも外部へ文章を送信し、利用料金が発生します。

コサイン類似度

2つのベクトルの向きの近さを計算します。

内積を、それぞれの長さの積で割ります。

内積 = $a[0] \times b[0] + a[1] \times b[1] + \dots$
長さ = 各成分の2乗の合計の平方根
類似度 = 内積 $\div$ (aの長さ $\times$ bの長さ)

同じ向きは1。ゼロベクトルは教材では0として扱います。

forで内積を計算

```
for i in range(len(a)):
    dot += a[i] * b[i]
    a_size += a[i] * a[i]
    b_size += b[i] * b[i]
```

`i` はリストの位置です。同じ位置の成分同士を掛けます。
教材では専用の数値計算ライブラリを使わず、処理を追えるようにします。

実習1：計算の穴埋め

`embeddings.py` の `cosine_similarity()` を完成させます。
コメントを外し、仮の `raise` を削除します。

ターミナルで `python` を起動して試します。

```
from embeddings import cosine_similarity
print(cosine_similarity([1, 0], [1, 0])) # 1
print(cosine_similarity([1, 0], [0, 1])) # 0
```

デモ用ベクトルの注意

デモは、指定した単語の出現回数を数値にしています。

- 外部のEmbeddingモデルを使わない
- 意味や言い換えを理解しない
- ベクトルの保存・比較の流れを観察するために使う
- 本物の意味検索の品質評価には使わない

デモとAPIモードの結果を混ぜて採点しません。

実習2：検索の準備

```
cd session06/exercise  
python init_db.py  
python build_index.py  
python main.py
```

APIモードは `.env` または環境変数を先に設定します。

`build_index.py` は、APIモードなら各断片を外部へ送ります。

初回は資料数を確認し、講師の案内に従って実行します。

DBに保存するもの

項目	保存する理由
本文	LLMへ渡すのは元の文章
embedding	次の検索で再利用する
embedding_model	質問と同じモデルか確かめる
ファイル名・見出し	根拠を確認する

ベクトルだけをLLMへ渡しても、この教材の回答は作れません。

実習3：同じ質問で比較

APIモードで次の質問を比較します。

1. 「SQLiteに保存するには？」
2. 「ページを閉じててもデータが残るのはなぜ？」
3. 「第7回の提出物は？」
4. 「来年度の試験日は？」

キーワード検索と意味検索で、選ばれた資料を記録します。
比較する上位件数と資料の版をそろえます。

スコアの読み方

類似度は、回答が正しい確率ではありません。

また、キーワード検索の点数と同じ尺度でもありません。

- 高いスコアでも、質問の答えが含まれない場合がある
- 固有名詞や記号はキーワード検索が役立つ場合もある
- 件数やしきい値は、評価用質問で検討する

この教材は正のスコアから上位件数を選ぶ単純な実装です。

資料を更新した場合

```
python init_db.py  
python build_index.py
```

再登録は、古いベクトルを消します。

モデルやデモ/APIモードを変えた場合も、再作成が必要です。

本文と古いベクトルの組み合わせで検索しないための仕組みです。

費用と規模

質問1回の意味検索では、通常、質問のEmbeddingと回答生成を呼びます。
資料のEmbedding作成にも別に費用がかかります。

この教材は少量の資料をPythonのループで全件比較します。
大規模な検索基盤の運用は、発展課題とします。
まずは検索精度と資料の整え方を学びます。

確認問題

1. 資料と質問に異なるEmbeddingモデルを使ってよい？
2. 類似度0.8は正答率80%？
3. 意味検索なら、根拠不足への対処は不要？
4. 資料を更新した後、何を作り直す？

提出物と次回

提出： `embeddings.py` と `worksheet.md` のGitHub URL。

2つの検索方式について、成功例と失敗例を1つずつ記録します。

APIを使えなかった場合は、その旨を明記し、講師の実演を観察します。

次回は、文章の検索に加えて、課題DBの値を取得します。